

RAPPORT DE PROJET

Détecter les pannes sur 5 lignes de production

Outils utilisés : MinIO (stockage), Airflow (automatisation), OpenMetadata (catalogue)

Étudiant : Stephane

Durée du projet : 8 jours

Environnement : Docker, Linux

Ce que j'ai construit : un système qui récupère les données de 5 machines d'usine, les nettoie, les range, et permet de savoir facilement d'où viennent les données et qui peut y accéder.

1. Comment j'ai organisé les données

1.1 Les 4 étapes (comme une chaîne de production)

J'ai rangé les données en 4 étapes, un peu comme sur une chaîne de montage : plus on avance, plus les données sont propres et prêtes à l'emploi.

Étape 1 - Brut (raw) : les fichiers CSV tels qu'ils arrivent de l'usine, sans y toucher. C'est la sauvegarde de référence.

Étape 2 - Nettoyé (staging) : je corrige les noms de colonnes, les types de données, j'enlève les erreurs.

Étape 3 - Prêt à l'emploi (curated) : les données sont calculées et prêtes pour l'analyse ou l'intelligence artificielle.

Étape 4 - Archive : les vieilles données gardées pour l'historique, avant suppression après 2 ans.

1.2 Les outils que j'ai choisis

MinIO : un espace de stockage sur mon ordinateur qui fonctionne comme Amazon S3 (le service de stockage cloud d'Amazon), mais en local et gratuit.

Apache Airflow : un outil qui exécute mes tâches automatiquement, dans le bon ordre, sans que j'aie à tout relancer à la main.

OpenMetadata : un genre de "carnet d'adresses" pour mes données. Ça permet de savoir ce qu'il y a dans chaque fichier sans avoir à l'ouvrir.

Parquet : un format de fichier plus rapide et plus léger que le CSV pour stocker des données.

1.3 Pourquoi cette organisation ?

Si demain je change d'outil (par exemple passer de MinIO à un vrai service cloud comme AWS), je n'ai presque rien à changer dans mon code. Chaque étape est indépendante des autres, donc si je casse quelque chose, je casse une seule chose à la fois, pas tout le pipeline.

2. Ce que j'ai concrètement fait

2.1 Les données de départ

J'ai reçu 5 fichiers CSV, un par ligne de production (LineA à LineE), soit environ 25 000 lignes de mesures au total (température, pression, etc.).

Problème rencontré : chaque fichier avait ses propres habitudes de nommage. Par exemple, une colonne s'appelait "Temperature" dans un fichier et "temperature" (minuscule) dans un autre. Certains fichiers avaient une colonne "elapsed_time" (temps écoulé), d'autres non.

Solution : j'ai créé un tableau de correspondance (un dictionnaire) qui dit "si tu vois ce nom, remplace-le par celui-ci". Ça harmonise tout automatiquement à l'étape 2 (staging).

2.2 L'automatisation avec Airflow

J'ai créé 2 automatisations (des "DAG", en langage Airflow) :

1. ingest_raw : récupère les CSV et les range dans MinIO, avec un système de dossiers organisé par année/mois/ligne de production. Pour le plus gros fichier (LineA, 10 000 lignes), je l'ai découpé en petits morceaux pour simuler un vrai flux de données en continu.
2. transform_to_staging : prend les fichiers bruts, corrige les noms de colonnes, transforme le format (CSV vers Parquet), et ajoute des informations utiles (quelle ligne, quand ça a été traité).

2.3 Le catalogue avec OpenMetadata

J'ai connecté OpenMetadata pour qu'il vienne "lire" automatiquement ce qu'il y a dans mes fichiers Parquet et créer des fiches descriptives : qui est responsable de la donnée, d'où elle vient, à quelle fréquence elle arrive. J'ai aussi documenté le chemin que suit chaque donnée (raw → staging → curated), ce qu'on appelle le "lineage".

3. Sécurité et règles d'accès

3.1 Qui a le droit de faire quoi

J'ai créé 3 profils d'utilisateurs différents, chacun avec des droits différents :

- data-analyst : peut seulement lire les données finales (curated), pas les modifier
- data-engineer : peut lire et écrire dans raw et staging (pour faire tourner les pipelines)
- admin : a tous les droits (pour la maintenance)

3.2 Suppression automatique et sécurité

J'ai mis en place des règles automatiques : les données sont archivées après 180 jours, puis supprimées après 2 ans si elles ne sont plus utiles. J'ai aussi activé le chiffrement (SSE-S3) sur les données sensibles, et des journaux qui notent qui a accédé à quoi et quand.

4. Difficultés rencontrées et solutions

Voici les principaux blocages rencontrés et comment je les ai résolus.

1. Fonction qui renvoie "rien" sans le dire — Ma fonction d'identification de ligne a créé un dossier vide "None" au lieu de signaler une erreur. Solution : ajout d'une vérification qui bloque et affiche une vraie erreur.
2. Fichiers en désordre dans le stockage — Des fichiers étaient mélangés (certains rangés par dossier, d'autres non), ce qui faisait planter le nettoyage. Solution : ménage complet + filtre strict sur les fichiers bien rangés.
3. Noms de colonnes différents entre fichiers — "Temperature" vs "temperature", colonnes manquantes selon les lignes. Solution : tableau de correspondance qui harmonise tout automatiquement.
4. OpenMetadata refusait de démarrer — Réglages de connexion à la base de données incorrects et version incompatible du moteur de recherche interne. Solution : bonne documentation + ajustement du fichier de configuration Docker.
5. Outil téléchargé mais cassé — Le client "mc" pour piloter MinIO ne pesait que 141 octets après téléchargement (au lieu de plusieurs Mo). Solution : ajout de l'option -L pour suivre la redirection.
6. Format de règles de suppression refusé — Mes règles ILM étaient écrites au format Amazon, alors que MinIO utilise son propre format JSON. Solution : réécriture dans le bon format.
7. Archivage cloud impossible en local — Le stockage "glacier" nécessite une infrastructure distante non disponible en local. Solution : suppression automatique après 2 ans à la place.
8. Erreur de structure dans la configuration — Le connecteur OpenMetadata-MinIO ne trouvait pas mes buckets car un champ était mal placé dans le JSON. Solution : déplacement du champ au bon niveau.

5. Bilan

Ce qui a été livré : schéma d'architecture (draw.io), 2 automatisations Airflow fonctionnelles, colonnes harmonisées automatiquement, catalogue OpenMetadata avec traçabilité, règles de sécurité (suppression auto, 3 comptes, chiffrement, journaux), documentation complète.

Ce que j'ai appris : quand quelque chose plante, il faut vérifier étape par étape plutôt que deviner. Une bonne partie du temps a été passée à comprendre les pannes plutôt qu'à écrire du nouveau code — c'est normal en data engineering. J'ai aussi compris qu'un simple souci de nom de colonne peut faire planter tout un pipeline si on ne le corrige pas tôt.

Pour aller plus loin : ajouter des tests automatiques, surveiller l'espace de stockage utilisé, et créer des alertes en temps réel quand une anomalie est détectée sur une ligne de production.